

Supplementary

1 Outline of LLM Prompt

We provide the outline of the LLM prompt for each task in BugLens. For simplified, we remove all examples (that used in few-shot in-context learning).

The `args` are the arguments for each case running, and the `callback` is the callback function for each prompt for the agentic design PKA.

1.1 Prompt for SecIA

```
I have a static analysis tool for Linux kernel drivers that produces many false positives when detecting:
1. Tainted Arithmetic Operations (integer overflows)
2. Tainted Loop Bound Conditions (infinite loops, unexpected iterations)
3. Tainted Pointer Dereferences (arbitrary memory access)
4. Buffer Overflow (out-of-bounds access)
5. Tainted length in copy_from_user (especially stack overflow)
Your Task:
1. Analyze the provided code snippet flagged by our tool
2. Assume:
  - Attackers cannot control the kernel
  - Tainted variables can be set to any value within their type range
  - All existing checks can be bypassed
  - Ignore any security checks in the code
3. Determine if the code represents:
  - Potential Bug: If the tainted variable can cause infinite loops, very large loops, or memory bugs in
    the current context
  - Normal Code: If the tainted variable usage doesn't lead to security issues within our scope
Provide a step-by-step explanation of your reasoning and classification.
=====
tainted_variable: {}
bug_detector: {}
source code
{}
=====

args:
- get_tainted_value
- get_bug_detector
- get_function
```

1.1.1 Schema-constrained summarization

```
Summarize our discussion, and respond with a <bug_eval> tag indicating whether the tainted variable can
lead to vulnerabilities:
currently we only consider (1) infinite/very-large loop, (2) out-of-bound access/buffer overflow/arbitrary
memory access;
for other types of bugs, you can say "normal code" or "not_a_bug"

<bug_eval>
  <tainted_var>tainted_var</tainted_var>
  <vulns>
    <vuln>
      <type>out_of_bound_access</type>
      <desc>Brief description of how the vulnerability occurs</desc>
    </vuln>
  </vulns>
</bug_eval>

For no vulnerability: <bug_eval>not_a_bug</bug_eval>
For potential vulnerability: <bug_eval>potential_bug</bug_eval>
If uncertain: <bug_eval>uncertain</bug_eval>
```

1.2 Prompt for ConA

This part we provide the prompt for the Constraint Assessor (ConA) task. These prompts already contain the design of Structured Analysis Guidance (SAG). Additionally, we also provide a list of functionalities of the agent Project Knowledge Agent (PKA) inside prompts, represented as `{AGENT PROMPTS HERE}`.

1.2.1 Step 1: Reachability Analysis

The following is the prompt for the step 1 of ConA task.

```
Identify the sink (the last line of the provided function context) and determine its preconditions.
Focus only on two types of preconditions:
1. Direct checks (dominate conditions) - Conditions that directly control if the sink executes
   Example: 'if (flag) { sink(tainted_var); }'
2. Early returns/bypasses (guard conditions) - Conditions that cause early returns or jumps that bypass the
   sink
   Example: 'if (tainted_var > 100) return; sink(tainted_var);'
Here, "tainted_var <= 100" is the precondition to reach the sink.
Ignore conditions that don't directly impact sink reachability.
=====
{AGENT PROMPTS HERE }
=====
sink variable
{}
sink context (the full context of the function):
{}
=====
Summarize in this format:
'''xml
<sink_precondi>
  <precondi>
    <type> dominate_condition </type>
    <condition> flag </condition>
    <dominated_sink> if(flag) sink(tainted_var) </dominated_sink>
  </precondi>
  <precondi>
    <type> guard_condition </type>
    <condition> tainted_var <= 100 </condition>
    <guard_bypass> if (tainted_var > 100) return/goto invalid_label; </guard_bypass>
  </precondi>
</sink_precondi>
'''
Note: Multiple preconditions combine with "AND" logic.

args:
- get_tainted_value
- get_function
callback:
- need_struct_def
- need_global_var_def
```

1.2.2 Step 2: Constraint Collection

The following is the step 2 of the prompt for the ConA task.

```
Help find range constraints for the tainted variable across the call chain.
=====
tainted_variable: {}
current callchain: {}
context of the sink:
{}
=====
Focus on finding all possible value range constraints for the tainted variable, without analyzing their
effectiveness.
Types of constraints to look for:
1. Validation: conditions that reject invalid ranges (e.g., if (tainted_var < 0) return -EINVAL;)
2. Sanitization: corrections applied to the value (e.g., tainted_var = min(tainted_var, 100);)
3. Type constraints: implicit limits from variable types (e.g., uint8_t limits to [0,255])
Important notes:
- Validations are transferable through operations (e.g., constraints on var = tainted_var + 1 apply to
  tainted_var)
- Sanitizations are not transferable
- Base your analysis only on the provided code, not prior knowledge
- Track the variable across different names in different contexts
=====
{AGENT PROMPTS HERE}
=====
Summarize your findings in this format:
-----
<range_constraints>
  <constraint>
    <type>validation|sanitization|type_constraint</type>
    <handler_func>function_name</handler_func>
    <context>relevant_code_snippet</context>
  </constraint>
</range_constraints>
-----
args:
- get_tainted_value
- get_call_chain
- get_function_first_part
callback:
```

- need_func_def
- need_caller
- need_struct_def
- need_global_var_def

1.2.3 Step 3: Constraint Effect Summarization

The following is the prompt of the step 3 of the ConA task.

Act as a program verifier/symbolic execution engine to infer precondition and postcondition pairs for constraints in code. Use these simple rules:

- The ****precondition**** is the path condition required to reach the constraint.
- The ****postcondition**** is the effect on the tainted variable (often shown as its valid range).

Consider all branches and early return "bypass" cases. For instance, given this sample function:

```
'''c
void check_tainted_value(int config, int other_config, int x){
    if (other_config == CHECK_SKIP)
        return;
    if (config == 1)
        assert(x > 0 && x < 100);
    else if (config == 2)
        assert(x > 0);
}
```

You must extract the following [precondition, postcondition] pairs:

1. Bypass Case
Precondition: other_config == CHECK_SKIP
Postcondition: x in (-inf, +inf)
2. Config 1 Case
Precondition: other_config != CHECK_SKIP && config == 1
Postcondition: x in (0, 100)
3. Config 2 Case
Precondition: other_config != CHECK_SKIP && config == 2
Postcondition: x in (0, +inf)
4. Default Case
Precondition: other_config != CHECK_SKIP && config != 1 && config != 2
Postcondition: x in (-inf, +inf)

```
=====
{AGENT PROMPTS HERE}
=====
```

Finally, output your analysis as XML in the following format:

```
'''xml
<range_constraint>
  <type>validation</type>
  <handler_func>check_tainted_value</handler_func>
  <condition_pairs>
    <pair>
      <precondi>other_config == CHECK_SKIP</precondi>
      <postcondi>x in (-inf, +inf)</postcondi>
      <context> if (other_config == CHECK_SKIP) return; </context>
    </pair>
    ...
    <pair>
      <precondi>other_config != CHECK_SKIP && config != 1 && config != 2</precondi>
      <postcondi>x in (-inf, +inf)</postcondi>
      <context> induced by the other branches </context>
    </pair>
  </condition_pairs>
</range_constraint>
'''
callback:
- need_func_def
- need_struct_def
- need_global_var_def
- need_caller
```

1.2.4 Step 4: Final Vulnerability Evaluation

The following is the prompt of the step 4 of the ConA task. (see §?? for details)

Evaluate if this bug is eliminated, not exploitable, or still vulnerable.
 Extract concrete values for "size of" or "length of" in constraints and bug conditions.
 Analyze whether the bug condition can be satisfied considering all constraints:

- Disregard constraints with postconditions that don't limit the tainted variable
- Disregard constraints for unrelated sinks
- For kernel-controlled conditions, determine reachability based on your knowledge
- If a precondition contains user-controlled variables, assume users can bypass it

A bug is "eliminated" only if:

- The postcondition restricts the tainted variable to a safe range that makes the bug impossible
- The precondition is always satisfied when the sink's precondition is true

If the postcondition isn't strong enough, assess exploitability assuming attackers can set any value.

```
=====
{AGENT PROMPTS HERE}
```

```

=====

callback:
- need_func_def
- need_struct_def
- need_global_var_def
- need_caller

```

1.2.5 Schema-constrained summarization

The following is the prompt for summarization of the ConA task, which is used to extract the final result of the vulnerability evaluation within a <final_res>.

```

Now Let's summarize our discussion, and respond in a <final_res> tag with the following format:
"still_a_bug", "eliminated", "likely_safe", "likely_unsafe", "not_exploitable" or "uncertain" within a <
final_res> tag, e.g., <final_res>still_a_bug</final_res>

```

1.3 Prompt for PKA

The following is the exact prompt that shown as AGENT PROMPTS HERE in the previous parts.

```

1  =====
2  First of all, you don't need to complete the task in your initial response. You can always ask for more
   information.
3
4  When you need additional details, use the following format. In this case, don't reach a conclusion
   immediately - instead, request the information you need to perform a thorough analysis once you
   receive my response.
5  --- request 1: ask for the function definition ---
6  You could ask me for the definition of the function. in this case, you could respond with the following:
7  <requests>
8    <request>
9      <name>need_func_def</name>
10     <args>
11       <arg>func_1</arg>
12       <arg>func_2</arg>
13     </args>
14   </request>
15 </requests>
16 --- request 2: ask for the struct definition ---
17 You could ask me for the definition of the structure. in this case, you could respond with the following:
18 <requests>
19   <request>
20     <name>need_struct_def</name>
21     <args>
22       <arg>struct_name_1</arg>
23       <arg>struct_name_2</arg>
24     </args>
25   </request>
26 </requests>
27 --- request 3: ask for the caller of the current function ---
28 You could ask me for the caller for the current function. (Note: you can only request one caller at a time)
29 in this case, you could respond with the following:
30 <requests>
31   <request>
32     <name>need_caller</name>
33     <args>
34       <arg>current_function_name</arg>
35     </args>
36   </request>
37 </requests>
38 --- request 4: ask for the definition of global variables ---
39 You could ask me for the definition of global variables. In this case, you could respond with the following
   :
40 <requests>
41   <request>
42     <name>need_global_var_def</name>
43     <args>
44       <arg>global_var_1</arg>
45       <arg>global_var_2</arg>
46     </args>
47   </request>
48 </requests>
49 =====

```